INTERVIEW PREPARATION SERIES

Java Control Flow
Interview Preparation

20 Questions on Conditional Statements, Nested Ifs, Loops & Nested Loops

ABOUT THIS DOCUMENT

A conversational, interview-toned deep dive into Java control flow — **20 questions across 4 core topics**, written for engineers with ~3 years of experience. Covers modern Java (Java 14–21) switch expressions, pattern matching, ConcurrentModificationException, stream trade-offs, and the mathematics of nested loop analysis.

TOPIC 01

Conditional Statements

QUESTION 01

“In Java, can you use a switch statement on a String? What are the rules and how does Java implement it internally?”

ANSWER

Yes — since Java 7. Java allows switching on `String`, `int`, `Integer`, `char`, `Character`, `byte`, `short`, `enum`, and (since Java 21) any reference type via pattern matching.

Internal implementation: the compiler translates a String switch into two steps: first, a switch on the `hashCode()` of the string, then an `equals()` check inside each case to guard against hash collisions. This is why String switch is O(1) average — but not O(1) worst case (collision possible).

```
// What you write:
switch (command) {
    case "start" -> launch();
    case "stop"  -> halt();
    default      -> unknown();
}

// What the compiler does internally (approximately):
switch (command.hashCode()) {
    case 109757538:  // "start".hashCode()
        if (command.equals("start")) launch();
        break;
    ...
}
```

Java 14+: switch expressions. Switch can now *return a value*, eliminating verbose break-and-assign patterns. **Java 21: pattern matching** for switch — `case String s when s.isEmpty()` — transforms the switch from a constant-matching tool into a full pattern-matching construct.

QUESTION 02

"What's the difference between && and & on booleans in Java? When would you deliberately choose &?"

ANSWER

&&: short-circuit logical AND. Right operand not evaluated if left is false.

& on booleans: eager logical AND. Both operands always evaluated. Same result, different side-effect semantics.

When to deliberately choose &: when the right operand has a side effect you *need* to happen regardless of the left.

```
// Both checks must run: counter must be incremented even if first fails
boolean valid = isFormatValid(input) & counter.increment();

// With &&: counter.increment() never runs if isFormatValid returns false
boolean valid = isFormatValid(input) && counter.increment();
```

This is rare but exists in auditing, metrics collection, and test frameworks where both evaluations must be recorded. The more common pattern is `&&` for null guards, short-circuit performance, and correctness.

Trap: `&` on non-booleans is bitwise AND. `5 & 3 = 1`. The operator's meaning depends entirely on the operand types — same syntax, radically different semantics.

QUESTION 03

"Java 14 introduced switch expressions. Walk me through what changed and why the new form is safer than the old switch statement."

ANSWER

```
// Old switch statement: fall-through by default, returns nothing
String result;
switch (day) {
    case MONDAY:
    case TUESDAY:
        result = "Early week";
        break;
    case FRIDAY:
        result = "End of week";
        break;
    default:
        result = "Mid week";
}

// New switch expression (Java 14+): no fall-through, returns value
String result = switch (day) {
    case MONDAY, TUESDAY -> "Early week";
    case FRIDAY          -> "End of week";
    default              -> "Mid week";
};
```

- **No fall-through by default:** arrow `->` cases don't fall through. Eliminates the most common switch bug.
- **Exhaustiveness:** compiler enforces that all cases are covered (on sealed types or enums). Missing a case is a compile error, not a silent bug.
- **Returns a value:** eliminates the mutable `result` variable pattern. The switch expression is an expression, usable anywhere.

- **yield:** use `yield value;` instead of `break value;` for multi-statement cases that need to return a value.

Java 21 extension: pattern matching allows `case Integer i when i > 0 -> ...` — guarded patterns eliminate entire classes of cascading instanceof checks.

QUESTION 04

“What does it mean that Java’s if condition must be a boolean? Show me a Java compile error that would silently compile in C.”

ANSWER

Java requires the if condition to be exactly `boolean` or `Boolean`. No implicit conversion from integer or pointer.

```
// This compiles silently in C (assignment inside condition)
int x = 5;
if (x = 0) { ... }   // Always false — classic C bug

// Java: compile error
int x = 5;
if (x = 0) { ... }   // Error: incompatible types: int cannot be converted
to boolean
```

Java caught a design flaw from C: the type system forces you to write `==` instead of `=` in conditions. This prevents the “assignment instead of comparison” bug entirely, by design.

The one exception: `if (b = someBoolean())` compiles because `b` is a boolean. Assigning a boolean inside an if condition is valid Java — but still a readability smell. Use with caution; document with a comment.

Autoboxing note: `Boolean b = null; if (b) { ... }` compiles but throws `NullPointerException` at runtime when unboxing. Null-check before using `Boolean` wrapper in conditions.

QUESTION 05

“Explain the ternary operator in Java. When is it dangerous, and why does it sometimes trigger autoboxing NPEs?”

ANSWER

Ternary operator: `condition ? expr1 : expr2` evaluates condition; returns expr1 if true, expr2 if false. Both branches must be type-compatible.

The autoboxing NPE trap:

```
Integer a = null;
int result = condition ? a : 0;   // NPE if condition is true
// Java unboxes 'a' to int before assigning to result
// Unboxing null Integer → NullPointerException
```

Why: when one operand is a primitive (`0` is `int`) and the other is a wrapper (`Integer a`), Java unboxes the wrapper to match types. Unboxing null → NPE. The error appears on an innocent-looking assignment line.

Fix: ensure both branches are the same type explicitly, or check for null first.

```
int result = condition ? (a != null ? a : 0) : 0;
// Or: use Optional, or null-safe utilities
```

Performance note: ternary is not inherently faster than if-else. JIT compiles them identically. Prefer if-else when the logic is complex; prefer ternary for simple value-selection one-liners.

TOPIC 02

Nested Ifs

QUESTION 06

“What is the guard clause pattern and why does Java code often prefer it over deeply nested ifs?”

ANSWER

Guard clauses are early returns (or throws) that handle invalid/edge states at the top of a method, leaving the happy path flat and unindented.

```
// Deep nesting: reader must track all conditions simultaneously
void process(Order order) {
    if (order != null) {
        if (order.isValid()) {
            if (order.hasItems()) {
                if (order.getCustomer().isActive()) {
                    // actual logic buried 4 levels deep
                }
            }
        }
    }
}

// Guard clauses: each failure exits immediately
void process(Order order) {
    if (order == null) throw new IllegalArgumentException("Order
required");
    if (!order.isValid()) return;
    if (!order.hasItems()) return;
    if (!order.getCustomer().isActive()) return;
    // actual logic at top level — easy to read
}
```

The refactored version reduces cyclomatic complexity, making the method easier to test (each guard is one unit test), easier to review (conditions are co-located with their errors), and easier to extend.

Java 21 variant: pattern matching in instanceof and switch reduces many if-null-cast chains to single-line guards.

QUESTION 07

“How does Java’s pattern matching for instanceof change how you write nested ifs? Show before and after.”

ANSWER

```
// Before Java 16: explicit cast after instanceof — three operations
Object obj = getMessage();
if (obj instanceof String) {
    String s = (String) obj;             // manual cast
    if (s.length() > 10) {
        System.out.println(s.toUpperCase());
    }
}

// Java 16+: pattern variable — one operation, no cast
if (obj instanceof String s && s.length() > 10) {
    System.out.println(s.toUpperCase());  // s is String, in scope, safe
}
```

Pattern matching eliminates the manual cast (a common source of ClassCastException if the check and cast get out of sync during refactoring) and enables the condition and usage to be in a single expression.

Java 21 switch pattern: extends this further, replacing entire if-else-instanceof chains:

```
// Replacing nested instanceof if-else:
String describe(Object obj) {
    return switch (obj) {
        case Integer i -> "Integer: " + i;
        case String s when s.isEmpty() -> "Empty string";
        case String s -> "String: " + s;
        case null -> "null";
        default -> "Other";
    };
}
```

QUESTION 08

“What is the cyclomatic complexity of a method and why does it matter in interviews and code reviews?”

ANSWER

Cyclomatic complexity counts the number of linearly independent paths through a method. Each `if`, `else if`, `for`, `while`, `do-while`, `case`, `catch`, ternary operator (`?:`), `&&`, `||` adds 1 to the count. Start counting from 1.

```
void process(int x, boolean flag) {     // CC = 1
    if (x > 0) {                        // CC = 2
        if (flag) {                     // CC = 3
            // do something
        }
    } else {                            // else doesn't add
        for (int i = 0; i < x; i++) {  // CC = 4
            System.out.println(i);
        }
    }
}  // Final CC = 4
```

- **CC 1–4:** simple, easy to understand and test.
- **CC 5–10:** moderate, review carefully.
- **CC > 10:** high risk, likely needs refactoring. SonarQube, Checkstyle, PMD flag this.

Why it matters: CC = minimum number of unit tests to achieve branch coverage. A CC of 15 means 15 independent paths — the method is almost certainly too long and doing too much. Google’s Engineering Practices recommend CC ≤ 10 per method.

TOPIC 03

Loops

QUESTION 09

“Compare Java’s for, for-each, while, and do-while. What can a for-each loop NOT do that a regular for loop can?”

ANSWER

- **for**: index-based. Use when you need the index, want to skip elements, iterate backward, or iterate two collections simultaneously.
- **for-each (enhanced for)**: iterates over any Iterable. Cleaner, prevents off-by-one errors. Cannot: access the current index, modify the list structure (ConcurrentModificationException), iterate backward, or process two collections in one loop.
- **while**: when the iteration count is unknown at the start. Condition checked before each iteration.
- **do-while**: body guaranteed to run at least once. Rare in Java — most naturally expressed as a while with an initial call.

```
// For-each CANNOT: modify while iterating
for (String s : list) {
    if (s.isEmpty()) list.remove(s);   // ConcurrentModificationException

// For-each CANNOT: access index
for (String s : list) {
    System.out.println(??? + ": " + s);   // no index available

// Must use traditional for or ListIterator
for (int i = 0; i < list.size(); i++) {
    System.out.println(i + ": " + list.get(i));
```

QUESTION 10

“What is ConcurrentModificationException and when does it fire? Does it have anything to do with threads?”

ANSWER

Misleading name: ConcurrentModificationException fires in single-threaded code when you structurally modify a collection while iterating it. Nothing to do with threads.

```
List<String> names = new ArrayList<>(Arrays.asList("Alice", "Bob",
"Carol"));
for (String name : names) {
    if (name.startsWith("B"))
        names.remove(name);   // ConcurrentModificationException
}
```

Mechanism: ArrayList's iterator tracks a `modCount` snapshot. Every structural change (add/remove) increments the list's `modCount`. Each `next()` call checks if the snapshot still matches. If not — fail-fast.

Correct fixes:

- `removeIf`: `names.removeIf(name -> name.startsWith("B"))`. Java 8+. Atomic, O(n).
- `Iterator.remove()`: the iterator's own remove method is safe because it updates modCount.
- `CopyOnWriteArrayList`: iteration is on a snapshot; modifications don't affect the running iterator. Use for infrequent writes, frequent reads.
- Collect to a separate list, remove after iteration.

QUESTION 11

"Explain labeled break and labeled continue in Java. C++ doesn't have these — what's the Java equivalent?"

ANSWER

Java allows labels on loop statements. `break label` exits the labeled loop; `continue label` skips to the next iteration of the labeled loop, bypassing any inner loops.

```
outer:
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        if (matrix[i][j] == target) {
            System.out.println("Found at " + i + "," + j);
            break outer;    // exits BOTH loops
        }
    }
}
// Execution continues here after break outer
```

Labeled continue:

```
outer:
for (int i = 0; i < 5; i++) {
    for (int j = 0; j < 5; j++) {
        if (skip_condition)
            continue outer;   // skips rest of inner loop, next outer
iteration
    }
}
```

C++ has no labeled break. Its equivalent: `goto label`, boolean flags, or extracting to a function and using `return`. Java's labeled break is cleaner and explicit about which loop is being exited.

QUESTION 12

"You call stream() instead of a for loop to process a list. When is that the wrong choice?"

ANSWER

Streams are powerful, but four situations favor loops:

- **Exception handling:** checked exceptions inside stream lambdas require wrapping or sneaky throws. `forEach(item -> { try { ... } catch (IOException e) { ... } })` is verbose and swallows context. A loop handles exceptions cleanly.
- **Mutable state:** streams are designed for stateless operations. Accumulating into a mutable local variable from inside a stream requires `AtomicInteger` or reduction — awkward. Loops mutate freely.
- **Early exit:** there is no labeled break in a stream. Finding a match requires `findFirst()anyMatch()takeWhile()` — sometimes less clear than a `break`.
- **Performance in tight loops:** for small collections, stream overhead (lambda allocation, spliterator creation) can exceed the loop's work. `IntStream.range(0, 10)` is slower than `for (int i = 0; i < 10; i++)` for trivial bodies. Benchmark before switching.

Rule: use streams for *data transformation pipelines* — filter, map, collect. Use loops for *imperative control flow* where break, continue, mutable state, or checked exceptions matter.

QUESTION 13

“What does the eof-check anti-pattern look like in Java I/O loops and why is it wrong?”

ANSWER

```
// Anti-pattern: check before read — can process last line twice
while (!reader.ready()) {    // 'ready' isn't EOF — wrong method entirely
    String line = reader.readLine();
    process(line);
}

// Also wrong: check after EOF using a flag
while (true) {
    String line = reader.readLine();
    if (line == null) break; // correct check, but awkward structure
    process(line);
}
```

Correct patterns:

```
// Pattern 1: readLine() returns null at EOF — assign and check
String line;
while ((line = reader.readLine()) != null) {
    process(line);
}

// Pattern 2: Java 8+ streams API
reader.lines().forEach(this::process);

// Pattern 3: Scanner
while (scanner.hasNextLine()) {
    process(scanner.nextLine());
}
```

The key: `readLine()` returns null at EOF — the null check *is* the EOF check. Any approach that checks for EOF without actually reading risks processing the same last element twice or missing it.

TOPIC 04

Nested Loops

QUESTION 14

“What is the time and space complexity of this nested loop? Explain every term.”

```
public List<int[]> findPairs(int[] arr, int target) {
    List<int[]> result = new ArrayList<>();
    for (int i = 0; i < arr.length; i++) {
        for (int j = i + 1; j < arr.length; j++) {
            if (arr[i] + arr[j] == target) {
                result.add(new int[]{arr[i], arr[j]});
            }
        }
    }
    return result;
}
```

ANSWER

Time: O(n²) — inner loop runs (n-1) + (n-2) + ⋯ + 1 = n(n-1)/2 times = Θ(n²).

Space: O(k) where k = number of pairs found (the result list). If k = O(n²) worst case (all pairs sum to target in a carefully constructed array), space is O(n²). Otherwise, O(1) auxiliary space for the loop variables.

Can we do better? O(n) time, O(n) space: put all elements in a HashMap. For each element x, check if target-x is in the map. Single pass. Trade space for time.

Follow-up the interviewer will ask: “What if the array has duplicates and you can use the same element twice?” Tests whether you understand the j = i+1 constraint.

QUESTION 15

“Why should you avoid calling an O(n) method inside an O(n) loop? Show a Java String example.”

ANSWER

```
// O(n^2) disguised as O(n):
String result = "";
for (String s : list) {       // n iterations
    result += s;              // String concatenation: O(length of result)
}
// Concatenation at iteration k: O(k * avg_length)
// Total: O(1 + 2 + ... + n) * avg_length = O(n^2 * avg_length)
```

Each `+=` on an immutable String creates a new String, copying all previous characters. For 10,000 short strings, this produces ~50 million character copies instead of ~500,000.

```
// Fix: StringBuilder — O(n) amortized
StringBuilder sb = new StringBuilder();
for (String s : list) {
    sb.append(s);             // O(1) amortized — no copy
}
String result = sb.toString(); // one copy at the end
```

Other common hidden-O(n) methods inside loops:

- `contains()` on ArrayList: O(n) scan. Use HashSet for O(1).
- `get(i)` on LinkedList: O(n) traversal. Use ArrayList or iterator.
- `Collections.sort()` inside a loop: O(n log n) per iteration = O(n² log n) total.

QUESTION 16

“Write a method to print a multiplication table using nested loops. Then tell me how to optimise it if it’s called millions of times.”

ANSWER

```
// Naive: compute on every call
void printTable(int n) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            System.out.printf("%4d", i * j);
        }
        System.out.println();
    }
}   // O(n^2) time per call, O(1) space
```

If called millions of times with the same n: precompute and cache.

```
private static Map<Integer, int[][]> cache = new HashMap<>();

int[][] getTable(int n) {
    return cache.computeIfAbsent(n, size -> {
        int[][] table = new int[size][size];
        for (int i = 0; i < size; i++)
            for (int j = 0; j < size; j++)
                table[i][j] = (i + 1) * (j + 1);
        return table;
    });
}
```

First call: O(n²) build. All subsequent calls: O(1) lookup. Memory: O(n²) per unique n.

Production consideration: if n is large or varies widely, use a bounded cache (Caffeine/Guava with max size) to prevent unbounded heap growth. Precomputation trades space for amortised time.

QUESTION 17

“You see this code in production. What’s wrong with it?”

```
for (int i = 0; i < employees.size(); i++) {
    for (int j = 0; j < departments.size(); j++) {
        if (employees.get(i).getDeptId() == departments.get(j).getId()) {
            System.out.println("Match: " + employees.get(i).getName());
        }
    }
}
```

ANSWER

Three problems:

1. **O(n×m) nested loop** for what is fundamentally a join operation. With 10,000 employees and 1,000 departments: 10 million iterations.
2. **Repeated list.get() and method calls**: `employees.get(i).getDeptId()` is called twice per inner iteration. Extract once outside the inner loop.
3. **Wrong equality check**: `==` on Integer/Long wrappers compares references, not values. For IDs that are `Integer` objects, IDs beyond 127 will use different cached objects and `==` returns false even when IDs match. Use `.equals()`.

```
// Fix: O(m) preprocessing, O(n) lookup
Map<Long, Department> deptById = departments.stream()
    .collect(Collectors.toMap(Department::getId, d -> d));

for (Employee emp : employees) {
    Department dept = deptById.get(emp.getDeptId());
    if (dept != null)
        System.out.println("Match: " + emp.getName());
}
```

QUESTION 18

“Analyze this nested loop. The inner loop’s bound is not n. Be precise.”

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

ANSWER

Inner loop runs i times for outer iteration i. Total stars: Σ_{i=1}^{n} i = n(n+1)/2 = **Θ(n²)**. This prints a right triangle of stars. Classic pattern for recognising the triangular number n(n+1)/2.

This exact pattern appears in: bubble sort comparisons, insertion sort operations, all-pairs distance calculations, lower-triangular matrix fills. Immediately knowing the sum is n(n+1)/2 ≈ n²/2 = Θ(n²) is a sign of mathematical fluency.

QUESTION 19

“How does Java’s parallel stream change nested loop behaviour? What are the correctness risks?”

ANSWER

```
// Sequential: deterministic order
list.stream().forEach(item -> process(item));

// Parallel: non-deterministic order, possible race conditions
list.parallelStream().forEach(item -> process(item));
```

Correctness risks with parallelStream() in nested iteration patterns:

- **Shared mutable state:** if `process()` modifies a shared list or counter, parallel execution causes races. Use `reduce()collect()Collectors.toList()` instead of mutating shared state.
- **Order-dependent logic:** if the inner step depends on results of the outer step, parallelism is unsafe. Outer-inner dependency must be eliminated first.

- **Overhead for small n:** parallelStream splits the work and joins threads. For n < ~10,000 elements with O(1) work per element, thread overhead exceeds the gain. `parallelStream` is useful for CPU-bound work on large data.

When parallelStream is safe: stateless operations (no shared mutable state), results collected with thread-safe collectors, and n is large enough to justify the overhead. **Benchmark before shipping.**

QUESTION 20

"This nested loop has an unusual exit pattern. What does it print for n=4 and what's the complexity?"

```
int count = 0;
outer:
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        count++;
        if (j == i) continue outer;
    }
}
System.out.println(count);
```

ANSWER

The `continue outer` jumps to the next iteration of the outer loop when `j == i`. For each i, the inner loop runs from j=0 to j=i (inclusive), then continues outer. Iterations per i: i+1.

Total count = Σ_{i=0}^{n-1}(i+1) = 1 + 2 + 3 + ⋯ + n = n(n+1)/2. For n=4: 10. Complexity: **Θ(n²)**, specifically n(n+1)/2 iterations.

What makes this question interesting: the labeled `continue` terminates the inner loop early (cutting it off at j=i), but the *total* work is still Θ(n²) because the cut-off pattern sums to the triangular number rather than to n². Knowing this separates engineers who count from those who reason mathematically.